

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

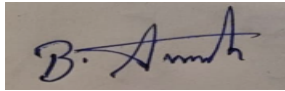
Assessment Tool Weightage: 20%

Code of Conduct:

I Avinash B (192511193)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read "B. Avinash", is written on a light-colored rectangular background.

Question 2: OCR Pipeline for Document Processing

Constraints: input images may be noisy and low contrast; batch processing required.

```
FUNCTION PreprocessImage(image):
    gray = ConvertToGrayscale(image)
    denoised = ApplyBilateralFilter(gray)           // removes noise, preserves text edges
    enhanced = ApplyCLAHE(denoised)                 // improves local contrast under uneven
lighting
    RETURN enhanced

FUNCTION SegmentTextRegions(enhanced_image):
    binary = AdaptiveThreshold(enhanced_image)      // handles varying illumination better than
a global threshold
    cleaned = MorphologicalClose(binary, kernel=(3,3)) // joins broken characters/words
    contours = FindContours(cleaned)
    text_regions = []
    FOR EACH contour IN contours:
        IF ContourAreaWithinTextRange(contour):
            text_regions.append(BoundingBox(contour))
    RETURN SortRegionsTopToBottomLeftToRight(text_regions)

FUNCTION ExtractText(image, text_regions):
    results = []
    FOR EACH region IN text_regions:
        roi = CropRegion(image, region)
        text = OCR_Engine(roi)                     // e.g., Tesseract
        results.append(text)
    RETURN JoinWithLineBreaks(results)

FUNCTION ProcessBatch(folder_path):
    output_results = {}
    FOR EACH file IN ListImageFiles(folder_path):  // batch-processing requirement
        image = ReadImage(file)
        IF image IS None:
            LogError(file, "unreadable file")
            CONTINUE
        enhanced = PreprocessImage(image)
        regions = SegmentTextRegions(enhanced)
        text = ExtractText(enhanced, regions)
        output_results[file] = text
        DisplayOrSave(file, text)
    RETURN output_results
```

Justification: bilateral filtering denoises while keeping text edges sharp; CLAHE and adaptive thresholding directly counter the stated low-contrast/uneven-illumination constraint (a single global threshold would fail on unevenly lit scans); the outer ProcessBatch loop with per-file error handling satisfies the batch-processing requirement without one corrupt file halting the whole run.

Question 6: Real-Time Object Detection Pipeline

Constraints: must process frames efficiently; handle multiple objects.

```
model = LoadPretrainedModel("model_weights")      // e.g., MobileNet-SSD / YOLO-tiny via
cv2.dnn

FUNCTION DetectObjects(frame):
    resized = Resize(frame, target_size)           // fixed small input size -- consistent,
fast inference
    blob = PreprocessForModel(resized)
```

```

    model.SetInput(blob)
    raw_detections = model.Forward()
    detections = []
    FOR EACH detection IN raw_detections:
        IF detection.confidence > CONFIDENCE_THRESHOLD:
            detections.append(detection)
    filtered = NonMaxSuppression(detections, iou_threshold=0.4) // removes duplicate overlapping
boxes
    RETURN filtered // -- correctly handles multiple,
possibly overlapping objects

FUNCTION DrawResults(frame, detections):
    FOR EACH det IN detections:
        DrawRectangle(frame, det.box)
        DrawLabel(frame, det.class_name, det.confidence, det.box)
    RETURN frame

FUNCTION MainLoop(video_source):
    cap = OpenVideoCapture(video_source)
    frame_count = 0
    last_detections = []
    WHILE cap.IsOpened():
        ret, frame = cap.Read()
        IF NOT ret: BREAK
        frame_count += 1
        IF frame_count % SKIP_INTERVAL == 0: // run inference on every Nth frame if
it's the bottleneck
            last_detections = DetectObjects(frame)
            output_frame = DrawResults(frame, last_detections) // still draws every frame using latest
known result
            Display(output_frame)
            IF UserPressedQuit(): BREAK
    cap.Release()

```

Justification: Non-Max Suppression is what correctly resolves multiple/overlapping detected objects into distinct boxes rather than duplicates. The frame-skip-with-reuse pattern lets inference run at a sustainable rate ('process frames efficiently') while the display loop still updates every frame using the latest available detections, keeping the visual output smooth even when the model can't run at full camera frame rate.

Question 9: Background Subtraction Pipeline

Constraints: handle dynamic background; reduce noise.

```

bg_subtractor = CreateBackgroundSubtractorMOG2(history=500, varThreshold=16, detectShadows=True)
// adaptive Gaussian-mixture model absorbs
gradual, dynamic background change

FUNCTION ProcessFrame(frame):
    blurred = ApplyGaussianBlur(frame, kernel=(5,5)) // reduces sensor noise before
subtraction
    fg_mask = bg_subtractor.Apply(blurred)
    fg_mask = RemoveShadowPixels(fg_mask) // MOG2 marks shadow separately --
discard it
    fg_mask = MorphologicalOpen(fg_mask, kernel=(3,3)) // removes small noise blobs
    fg_mask = MorphologicalClose(fg_mask, kernel=(7,7)) // fills gaps within real moving objects
    RETURN fg_mask

FUNCTION HighlightMovingObjects(frame, fg_mask):
    contours = FindContours(fg_mask)
    FOR EACH contour IN contours:
        IF ContourArea(contour) > MIN_AREA_THRESHOLD: // ignore tiny noise-driven contours

```

```

        box = BoundingRect(contour)
        DrawRectangle(frame, box)
    RETURN frame

FUNCTION MainLoop(video_source):
    cap = OpenVideoCapture(video_source)
    WHILE cap.IsOpened():
        ret, frame = cap.Read()
        IF NOT ret: BREAK
        fg_mask = ProcessFrame(frame)
        output = HighlightMovingObjects(frame, fg_mask)
        Display(output)
        IF UserPressedQuit(): BREAK
    cap.Release()

```

Justification: MOG2 is chosen over a static/single background frame specifically because its per-pixel Gaussian-mixture model adapts to gradual scene change (lighting drift, swaying trees), directly satisfying the dynamic-background constraint. Blurring before subtraction plus morphological opening/closing and an area threshold afterward together suppress noise at two different stages of the pipeline rather than relying on a single filter.

Question 11: Multi-Image Batch Processing System

Constraints: batch processing; efficient memory usage.

```

FUNCTION ProcessSingleImage(file_path):
    image = ReadImage(file_path)                                // load one image at a time -- never
    holds the whole dataset in memory
    IF image IS None:
        LogError(file_path)
        RETURN None
    preprocessed = Preprocess(image)                            // e.g., resize, denoise, normalize
    features = DetectFeatures(preprocessed)                      // task-specific
    detector/keypoints/edges
    RETURN {file: file_path, features: features}

FUNCTION ProcessBatch(folder_path, batch_size):
    file_list = ListImageFiles(folder_path)
    results_summary = []
    FOR i FROM 0 TO Length(file_list) STEP batch_size:          // process in fixed-size chunks, not
    all at once
        batch = file_list[i : i + batch_size]
        batch_results = []
        FOR EACH file IN batch:
            result = ProcessSingleImage(file)
            IF result IS NOT None:
                batch_results.append(result)
                StoreResult(result)                               // persist to disk immediately
            results_summary.append(Summary(batch_results))
            FreeMemory(batch_results)                             // release this batch once stored --
    bounds peak memory
    RETURN results_summary

```

Justification: processing and immediately persisting one bounded batch at a time — rather than loading every image into memory up front — keeps peak memory usage constant regardless of how large the input folder is, directly satisfying the efficient-memory-usage constraint, while the outer chunked loop still guarantees the entire folder is processed (the batch-processing requirement).

Question 14: Text Detection in Natural Scene Images

Constraints: handle varying backgrounds; improve detection accuracy.

```

FUNCTION PreprocessImage(image):
    gray = ConvertToGrayscale(image)
    enhanced = ApplyCLAHE(gray)           // normalizes contrast across varied
natural-scene lighting
    RETURN enhanced

FUNCTION DetectTextRegions(image, enhanced_image):
    candidate_regions = MSER_Detect(enhanced_image) // Maximally Stable Extremal Regions -
- robust to background clutter
    filtered_regions = []
    FOR EACH region IN candidate_regions:
        IF SatisfiesTextGeometryRules(region): // aspect ratio, size, stroke-width
consistency
            filtered_regions.append(region)
    merged_regions = MergeNearbyRegions(filtered_regions) // group individual characters into
text lines/words
    RETURN merged_regions

FUNCTION HighlightText(image, text_regions):
    FOR EACH region IN text_regions:
        DrawRectangle(image, region.bounding_box)
    RETURN image

FUNCTION Main(image_path):
    image = ReadImage(image_path)
    enhanced = PreprocessImage(image)
    text_regions = DetectTextRegions(image, enhanced)
    output = HighlightText(image, text_regions)
    Display(output)

```

Justification: MSER is specifically chosen over plain edge/contour detection because it reliably finds text-like blobs even against cluttered, varying natural-scene backgrounds — directly addressing that constraint. The geometry/stroke-width filtering step then removes non-text MSER regions (rejecting false positives from background texture), and CLAHE contrast normalization upstream improves detection accuracy under the varied lighting conditions typical of natural scene photos.